Métodos de Búsqueda en Sokoban y 8 Puzzle

TP1 — Sistemas de Inteligencia Artificial

Agentes, Búsqueda No Informada, Búsqueda Informada y Heurísticas

ITBA · 2025

Índice

1 El Problema: Sokoban

2 Agente y Ambiente

3 Formalización del Problema

4 Algoritmos Implementados

5 Heurísticas

6 Resultados Esperados

7 Resultados Obtenidos

8 Análisis Comparativo

9 Conclusiones

El 8 puzzle como problema de búsqueda

Formulación del espacio de estados, análisis algorítmico y diseño de heurísticas

Algunas definiciones y formulación del problema

-Dado un arreglo (lista) de enteros, decimos que el par ordenado (i, j) con i < j es una *inversión* si y solo si a_i < a_j.

Ejemplo:
1 3 4 2 5
Las posiciones (2, 4) y (3, 4) son *inversiones* del arreglo dado.

-Llamamos *tablero aplanado* al arreglo de enteros asociado a un tablero que se forma al recorrer el tablero en 'row-major order' (de izquierda a derecha, de arriba hacia abajo).

-Estado: s ∈{0,...,8}^9 (una permutación de números, donde 0 es la casilla vacía).
-Modelo de transición: δ(s, a) 'swappea' al o con un vecino ortogonal.
-Conjunto de acciones posibles: {U, D, L, R} (según posición del 0).
-Costo: costo de 1 uniforme para toda acción.

Invariante del 8 puzzle

Lema: la *paridad* de la cantidad de *inversiones* del tablero aplanado es **invariante** (lamentablemente no generaliza por completo para mayores dimensiones).

2 ‘clases de equivalencia’: estados iniciales con cantidad par de inversiones y cantidad impar.

Con la definición dada en el enunciado de ‘tablero resuelto’, siempre se podrá llegar a una solución!

Cantidad total de estados: 9!.
Estados alcanzables desde cierto estado inicial: 9!/2 (debido a las 2 clases de equivalencia existentes).

Podemos decir entonces que dos estados son *alcanzables* si y solo si tienen la **misma paridad de cantidad de inversiones**.

Un poco de teoría de grupos: hay 8 soluciones posibles, y son exactamente la órbita del tablero canónico bajo el grupo de simetrías del cuadrado D4!

Primera heurística: casillas ‘mal colocadas’

Sea G el conjunto de las 8 soluciones posibles ya mencionadas.
Para cada objetivo $g \in G$, contamos cuántas posiciones difieren entre s y g, y tomamos el mínimo.
$h_1(s) = \min_\{g \in \mathscr{G}\} |\{ t \in \{1,\ldots,8\} : s[t] \neq g[t] \}$
Es claro que h1 es admisible; cada movimiento reposiciona exactamente una casilla a lo sumo, por lo que reduce h1 en a lo sumo 1. Esto implica que h1 nunca sobreestima el costo real, así que es admisible.

Segunda heurística: distancia manhattan

Tomando la misma definición de G que en la heurística anterior:
Para cada objetivo g en G, sumamos las distancias Manhattan de cada ficha t, y nos quedamos con el mínimo.

$$h_2(s) = \min_{g \in \mathscr{G}} \sum_{t=1}^{8} \left(|r_t(s) - r_t(g)| + |c_t(s) - c_t(g)| \right)$$

donde r_t y c_t representan la fila y columna de la ficha t respectivamente.
Esta heurística también es admisible.
En particular, podemos afirmar que *domina* a h1.

Método de búsqueda elegido

Usaríamos A* con h2 como heurística (ya vimos que domina a h1). Vimos que es admisible, así que tiene una ‘poda’ necesariamente mejor (expande menos nodos)!

Ejercicio 2

Métodos de Búsqueda en Sokoban

ITBA · 2025

El Problema: Sokoban

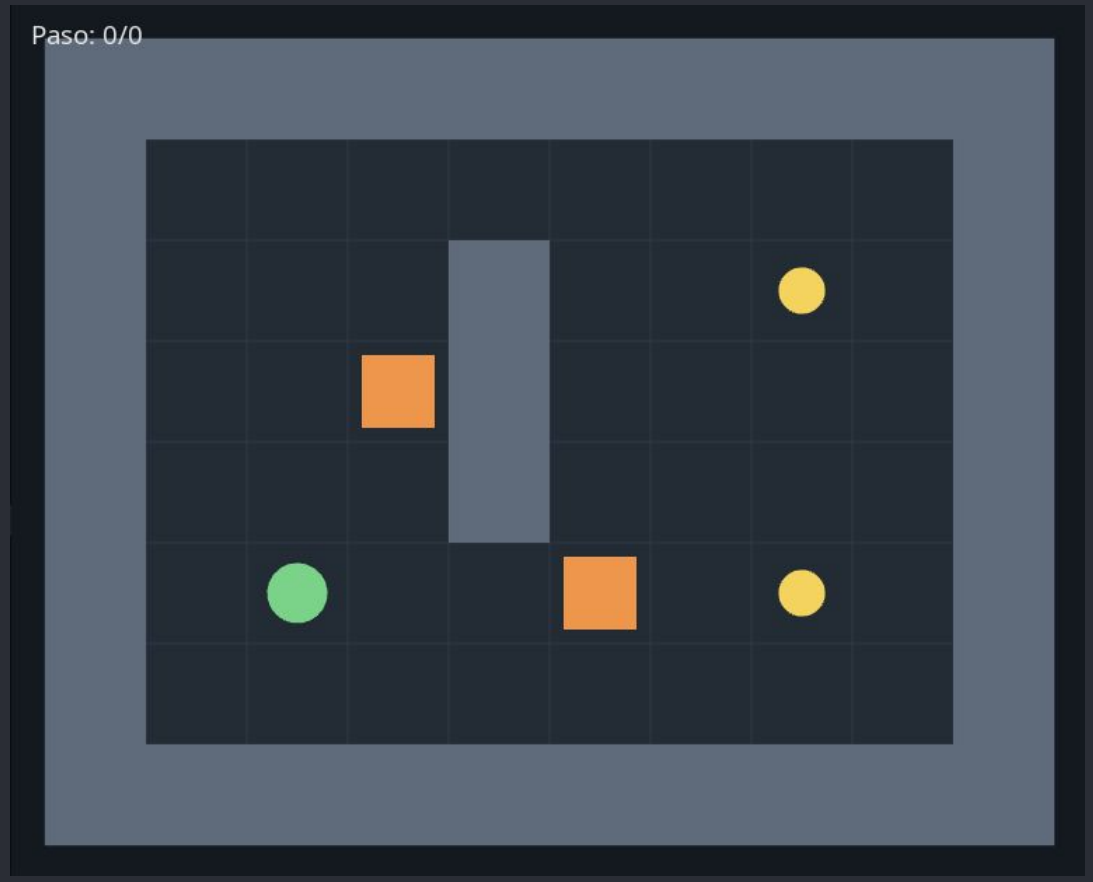
Desafio

→ Reubicación de cajas a las metas

Reglas

→ Las cajas solo pueden empujarse
→ Una caja en esquina puede quedar atrapada
→ El orden de las acciones puede ser determinante

Ejemplo



Agente y Ambiente

Caracterización del Agente

Propiedad	Valor
Tipo	Agente de búsqueda
Percepción	Estado completo del tablero
Acciones	Movimientos U / D / L / R
Objetivo	Todas las cajas en metas
Performance	Movimientos para resolver

Caracterización del Ambiente

Propiedad	Valor
Observable	Completamente observable
Determinista	Una acción determina un resultado único
Episódico	Secuencial (decisiones interdependientes)
Estático	No cambia mientras el agente piensa
Discreto	Tablero y acciones finitas
Agentes	Agente único

Ambiente completamente observable · determinista · secuencial · estático · discreto → ideal para búsqueda clásica

Formalización del Problema: Estructura de estado

Clase State - Dinamico

→ Posición del jugador: (fila, columna)
→ Posiciones de las cajas: frozenset{(f,c), ...}

Clase Board - Estatico

→ Posiciones de paredes
→ Posiciones de las metas
→ Dimensiones del tablero (filas × columnas)

```
# Estado inicial — Level 1
State(
    player = (5, 5),
    boxes  = frozenset({
        (2, 4),
        (4, 4),
        (5, 6)
    })
)
```

Formalización del Problema: Estructura de estado

Espacio de estados

Sea L el conjunto de todas las celdas libres (transitables, no paredes) del tablero, y n la cantidad total de cajas. Definimos el espacio de estados como:

$$S = \{(p, B) \mid p \in L, B \subset L, |B| = n, p \notin B\}$$

p: Coordenada bidimensional de la posición del jugador.

B: Subconjunto de coordenadas que contienen las cajas.

Restricción: El jugador no puede superponerse a una caja ($p \notin B$).

Tamaño del espacio de estados

L = celdas libres $\longrightarrow$ $L \times C(L, N)$ estados posibles
Level 1 (46 celdas, 3 cajas) $\approx$ 74,000 estados
BFS expande 497,706 nodos $\longrightarrow$ el grafo tiene ciclos

Función Objetivo

El estado es META si y sólo si todas las posiciones de cajas coinciden con las metas.

```
state.boxes == board.goals
```

Observación importante

No importa que caja está en que meta.
Solo importa que todas las metas estén cubiertas. El frozenset captura esto.

Formalización del Problema

Acciones y Función de Transición

Espacio de Acciones

A = { ARRIBA, ABAJO, IZQUIERDA, DERECHA }

→ No siempre disponibles — dependen del estado actual.

→ Costo uniforme: cada acción tiene costo = 1.

Función de Transición

T : Estado × Acción → Estado'

```
nueva_pos = jugador + (dx, dy)
si nueva_pos es pared:
    acción inválida
si nueva_pos tiene caja:
    nueva_caja = nueva_pos + (dx, dy)
    si nueva_caja es pared/caja:
        acción inválida
    si no: mover caja → nueva_caja
mover jugador → nueva_pos
```

Algoritmos Implementados

Vista general

Categoría	Algoritmo	Estrategia
No Informado	BFS	Cola FIFO — nivel a nivel
No Informado	DFS	Pila LIFO — profundidad primero
No Informado	DLS	DFS con límite fijo de prof.
No Informado	IDDFS	DFS iterativo por profundidad
Informado	A*	f = g + h (costo + heurística)
Informado	Greedy	f = h (solo heurística)

16 variantes en total: BFS · DFS · DLS · IDDFS · A(h1, h2, h1+p, h2+p, h2+dl, h2+p+dl) · Greedy(h1, h2, h1+p, h2+p, h2+dl, h2+p+dl)*

BFS — Búsqueda por Anchura

Estructura de datos

Cola FIFO — explora nodo a nodo por nivel de profundidad

Garantías

✓ Completo — siempre encuentra solución si existe
✓ Óptimo — primer camino = el más corto

Debilidad

✗ Memoria exponencial — guarda todos los nodos del nivel actual en la cola

Conclusión

Óptimo y completo, pero no escala bien.
Sirve de baseline para comparar el resto.

Nivel	Movimientos	Nodos Exp.	Tiempo
Level 1	39 ✓	497,706	2.79 s
Level 3	40 ✓	1,424,013	8.17 s
Level 7	24 ✓	318,815	1.83 s
Level 9	29 ✓	16,199,668	208.8 s
Level 10	26 ✓	1,676,232	10.75 s
Level 11	45 ✓	21,315,049	260.3 s

BFS siempre encuentra la solución. En nivel 11 tarda 260s y expande 21 millones de nodos.

DFS · DLS · IDDFS

DFS
Depth-First Search

Pila LIFO, profundidad primero.
Límite práctico = 3000.

✓ Poca memoria
✗ NO óptimo
✗ Timeout en nivel 11

Nivel 1:
Óptimo: 39 movs
DFS: 2,998 movs (77× peor)

DLS
Depth-Limited Search

DFS con límite de profundidad explícito. Si el límite < prof. óptima → no encuentra solución.

✓ Controla uso de memoria
✗ NO completo
✗ NO óptimo
✗ Timeout en nivel 11

Mismo comportamiento que DFS en nuestros experimentos.

IDDFS
Iterative Deepening DFS

Repite DFS aumentando el límite: hasta encontrar la solución óptima.

✓ Óptimo y completo
✓ Poca memoria (como DFS)
✗ Repite mucho trabajo

Nivel 1: 89.9 s (17.5M nodos)
Nivel 3: 307.9 s (59.8M nodos)
Nivel 9: TIMEOUT >500 s
Nivel 11: TIMEOUT >500 s

A*

Función de evaluación

$$f(n) = g(n) + h(n)$$

g(n) = costo real desde el inicio
h(n) = estimación del costo restante

Garantías (con h admisible)

✓ Completo
✓ Óptimo — garantizado si h nunca sobreestima

Variantes implementadas

astar_h1 → usa H1
astar_h2 → usa H2
astar_h1_player → H1 + jugador
astar_h2_player → H2 + jugador
astar_h2_deadlock → H2 + detección
astar_h2_player_deadlock → H2+jugador+detección

Algoritmo	Nivel 1	Nodos Exp.	Tiempo
BFS (base)	39 movs	497,706	2.79 s
A* H1	39 movs	229,732 (−54%)	2.04 s
A* H2	39 movs	190,575 (−62%)	1.97 s
A* H1+player	39 movs	178,878 (−64%)	1.79 s
A* H2+player ★	39 movs	147,823 (−70%)	1.73 s

Con H2+player, A* expande 70% menos nodos que BFS en Level 1.
En Level 9 la mejora llega al 97%.

Greedy

Función de evaluación

f(n) = h(n) (ignora g)

Solo mira la heurística.
Siempre va hacia el estado que parece más cercano a la meta.

Ventajas

✓ Extremadamente rápido
✓ Pocos nodos expandidos

Desventajas

✗ NO óptimo — puede quedar en óptimos locales
✗ Puede fallar en laberintos con trampas

Trade-off (Level 1)

Greedy H1+player: 47 movs | 365 nodos | 0.03 s

Algoritmo	Nivel 1	Nodos Exp.	Tiempo
Greedy H1	52 movs	13,567	0.12 s
Greedy H2	65 movs	9,831	0.11 s
Greedy H1+player	47 movs	365	0.03 s
Greedy H2+player	52 movs	573	0.03 s
— vs A* H2+player —	39 movs	147,823	1.73 s

Heurísticas — H1 y H2 (Admisibles)

H1 — Suma de Distancias Mínimas

Para cada caja sin ubicar, calcular la distancia Manhattan a la meta libre más cercana.

$$h1 = \Sigma \ \min\{ \text{manhattan}(caj_i, \text{meta}\square) \}$$

¿Por qué es admisible?

Nunca sobreestima — en el mejor caso, cada caja se mueve en línea recta a su meta más cercana.

Problema:
Puede asignar dos cajas a la misma meta → subestima más de lo necesario.

H2 — Asignación Óptima

Encuentra la asignación biyectiva (1 a 1)

$$h2 = \min_{\sigma \in perm \ i} \ \Sigma \ \text{manhattan}(caj_i, \text{meta}\sigma(i))$$

¿Por qué es admisible?

Ídem H1. Cada meta tiene su caja asignada
mejor cota inferior.

Propiedades clave:
H2 ≥ H1 siempre → H2 es más informada
H2 expande menos nodos que H1
Costo: O(N!) — práctico para pocas cajas

Heurísticas — H3, H4 y H5

H3 y H4 — Con Distancia del Jugador (Admisibles)

El jugador también necesita moverse hasta la primera caja antes de empezar a empujar.

h3 = h1 + max(0, dist(jugador→caja) − 1)
h4 = h2 + max(0, dist(jugador→caja) − 1)

Se resta 1: el último paso del jugador
ya empuja la caja (contado en h1/h2).

Efecto en A*:
Nivel 1: A*-H2 → 190K nodos
A*-H4 → 147K nodos (−22%)
Nivel 3: A*-H2 → 482K nodos
A*-H4 → 335K nodos (−30%)

H5 — Detección de Deadlocks (No Admisible)

Si una caja llega a una dead position nunca podrá alcanzar ninguna meta → estado inútil.

Implementación:
BFS hacia atrás desde cada meta. Las posiciones inalcanzables son dead positions (estáticas).

¿Por qué no es admisible?
Puede marcar estados como bloqueados cuando técnicamente son recuperables.

Resultado en experimentos:
Sin impacto medible — mismos nodos que sin detección. Nuestros niveles no tienen muchas dead positions estáticas; los deadlocks son dinámicos (cajas bloqueándose entre sí).

Resultados Esperados

Hipótesis antes de ejecutar

Completitud y Optimalidad

→ BFS, IDDFS, A* siempre encuentran la solución óptima
→ DFS/DLS darán soluciones muy largas
→ Greedy será rápido pero subóptimo

Eficiencia (nodos expandidos)

→ A* con H2 expandirá muchos menos nodos que BFS.

→ Agregar la distancia del jugador (H3/H4) mejorará H1/H2.

→ Greedy expandirá órdenes de magnitud menos que A*.

→ IDDFS explorará muchos nodos repetidos.

Escalabilidad

→ BFS e IDDFS fallarán en niveles muy complejos (memoria/tiempo).

→ DFS fallará en niveles ramificados.

→ A* con buenas heurísticas será el más robusto.

Detección de Deadlocks (H5)

→ Esperábamos una mejora significativa en todos los niveles.

→ Niveles con pasillos angostos y esquinas deberían beneficiarse más.

Resultados Obtenidos — Level 1

3 cajas · 8×10 · referencia principal

Algoritmo	Movimientos	Nodos Exp.	Nodos Gen.	Tiempo
BFS (base)	39 ✓	497,706	1,570,401	2.79 s
DFS / DLS	2,998 ✗ (77×)	505,630	1,457,058	2.33 s
IDDFS	39 ✓	17,554,026	48,046,830	89.87 s
A* H1	39 ✓	229,732 −54%	730,982	2.04 s
A* H2	39 ✓	190,575 −62%	607,143	1.97 s
A* H1+player	39 ✓	178,878 −64%	571,924	1.79 s
A* H2+player ★	39 ✓	147,823 −70%	473,048	1.73 s
Greedy H1	52 (1.3×)	13,567	42,685	0.12 s
Greedy H1+player	47 (1.2×)	365	1,220	0.03 s
Greedy H2+player	52 (1.3×)	573	1,908	0.03 s

Resultados Obtenidos — Niveles Difíciles

Level 9 (4 cajas · 10×10) y Level 11 (4 cajas · 12×11 con vallas)

Level 9

Algoritmo	Movs	Nodos Exp.	Tiempo
BFS	29 ✓	16,199,668	208.8 s
DFS / DLS	2,996 ✗	28,958,580	~400 s
IDDFS	—	—	TIMEOUT
A* H2	29 ✓	823,180	20.2 s
A* H2+player ★	29 ✓	519,921	16.0 s
Greedy H1	34	123	0.03 s
Greedy H2+player	59	499	0.03 s

Level 11 (más difícil)

Algoritmo	Movs	Nodos Exp.	Tiempo
BFS	45 ✓	21,315,049	260.3 s
DFS / DLS	—	—	TIMEOUT
IDDFS	—	—	TIMEOUT
A* H2	45 ✓	2,383,384	49.9 s
A* H2+player ★	45 ✓	1,540,036	37.6 s
Greedy H1	89	8,861,281	108.5 s
Greedy H2+player ★	67	1,869	0.05 s

En Level 11: Greedy H1 explora 8.8M nodos (108s) mientras Greedy H2+player explora solo 1,869 (0.05s) → misma familia, 440× menos nodos

Análisis — Completitud por Nivel

Algoritmo	Lvl 1–6	Lvl 7	Lvl 8	Lvl 9	Lvl 10	Lvl 11	Óptimo
BFS	✓	✓	✓	✓	✓	✓	✓
DFS	✓	✓	✓	✓	✓	TIMEOUT	✗
DLS	✓	✓	✓	✓	✓	TIMEOUT	✗
IDDFS	✓	✓	✓	TIMEOUT	✓	TIMEOUT	✓
A* (todas)	✓	✓	✓	✓	✓	✓	✓
Greedy (todas)	✓	✓	✓	✓	✓	✓	✗

Observaciones clave

→ BFS es el único que siempre termina (hasta Level 11), aunque tarde 260 segundos
→ DFS/DLS fallan en Level 11 porque el espacio de búsqueda es demasiado profundo y ramificado
→ IDDFS falla en Level 9 y 11 — repite demasiado trabajo en espacios con alto factor de ramificación
→ A* y Greedy (con cualquier heurística) completan todos los niveles

Análisis — Optimalidad de la Solución

Algoritmo	Lvl 1	Lvl 5	Lvl 7	Lvl 8	Lvl 9	Lvl 11	Óptimo
Óptimo (ref)	39	9	24	26	29	45	—
BFS	39 1.0×	9 1.0×	24 1.0×	26 1.0×	29 1.0×	45 1.0×	✓
IDDFS	39 1.0×	9 1.0×	24 1.0×	26 1.0×	—	—	✓
A* H2+player	39 1.0×	9 1.0×	24 1.0×	26 1.0×	29 1.0×	45 1.0×	✓
DFS / DLS	2998 77×	203 22×	742 31×	1670 64×	2996 103×	TIMEOUT	✗
Greedy H1	52 1.3×	9 1.0×	26 1.1×	26 1.0×	34 1.2×	89 2.0×	✗
Greedy H2+player	52 1.3×	9 1.0×	40 1.7×	28 1.1×	59 2.0×	67 1.5×	✗

→ DFS es gravemente subóptimo: en Level 1 da soluciones 77× más largas que el óptimo
→ Greedy es solo levemente subóptimo (1.1× – 2×) — razonable para su velocidad
→ En niveles simples, Greedy H1 a veces coincide con el óptimo

Análisis — Impacto de las Heurísticas en A*

Reducción de nodos expandidos respecto a BFS (baseline = 100%)

Algoritmo	Level 1	Level 3	Level 7	Level 9
BFS (base)	497,706 (100%)	1,424,013 (100%)	318,815 (100%)	16,199,668 (100%)
A* H1	229,732 (−54%)	599,289 (−58%)	20,828 (−93%)	1,445,134 (−91%)
A* H2	190,575 (−62%)	482,546 (−66%)	16,550 (−95%)	823,180 (−95%)
A* H1+player	178,878 (−64%)	421,112 (−70%)	12,508 (−96%)	909,811 (−94%)
A* H2+player	147,823 (−70%)	335,332 (−76%)	10,070 (−97%)	519,921 (−97%)

→ **A mayor complejidad del nivel, mayor es el beneficio RELATIVO de las heurísticas**
→ En Level 9, A*-H2+player expande 31× menos nodos que BFS
→ H2 siempre supera a H1 (asignación óptima vs. asignación greedy)
→ La distancia del jugador (H3/H4) reduce otro 10-30% adicional sobre H1/H2

Análisis — Fenómeno Interesante: Greedy H1 vs H2 en Level 11

El problema de H1 en Greedy

H1 es más barata de calcular por nodo que H2, pero guía PEOR la búsqueda.

Greedy H1 en Level 11:

→ 89 movimientos
→ 8,861,281 nodos expandidos
→ 108.5 segundos

Greedy H2 en Level 11:

→ 56 movimientos
→ 20,041 nodos expandidos (440× menos)
→ 0.26 segundos (418× más rápido)

Conclusión:

H2 es más costosa por nodo pero reduce tan drásticamente el espacio que en total es mucho más rápida.

Greedy en Level 11 — todos los resultados

Heur.	Movs	Nodos	Tiempo
H1	89	8,861,281	108.5s
H2	56	20,041	0.26s
H1+player	72	793,752	9.74s
H2+player	**67**	**1,869**	**0.05s**

La calidad de la heurística determina si el algoritmo escala o no, más que la elección entre A* y Greedy.

Detección de deadlocks (H5):

Sin impacto medible en ningún nivel.

Los deadlocks en nuestros niveles son dinámicos, no estáticos.

Conclusiones

1

La heurística importa más que el algoritmo base

A*-H2+player expande 97% menos nodos que BFS en Level 9. La calidad de h(n) determina la escalabilidad.

2

Greedy es sorprendentemente efectivo en Sokoban

Soluciones 10%-100% más largas que el óptimo, pero 100×-1000× más rápido. H2+player es la mejor combinación.

3

Los algoritmos no informados no escalan

BFS: viable con tiempo. IDDFS: impracticable en niveles difíciles. DFS/DLS: soluciones inutilizables.

4

H2 supera a H1 a pesar de ser más costosa por nodo

En Level 11 Greedy-H2 es 440× más eficiente que Greedy-H1. La mejor guía compensa el costo computacional.

5

La detección estática de deadlocks necesita refinamiento

Sin impacto en nuestros niveles. Deadlock dinámico (cajas bloqueándose mutuamente) sería más poderoso.